VeriSight: Master Q&A; Teleprompter & Defense Guide

Speaker Prep & Anticipated Audience Questions with Human-Toned Responses

Authors: Patel Kittu & Vinod Nikhil Monterio • Mirafra Technologies

---

How to use this guide: Below are 40 high-probability questions compiled across 8 categories that engineering directors, verification architects, and evaluators are most likely to ask during your presentation. Each answer is drafted in **natural, first-person spoken English**. You can look directly at this teleprompter during online meetings and read smoothly without sounding robotic.

1. High-Level Vision, Business Value & ROI

Q1. In simple terms, what exactly is VeriSight and what core problem does it solve?

Delivery Cue: *Keep eye contact, smile, and emphasize 'in under 15 seconds'.*

Spoken Answer: "VeriSight is an autonomous AI copilot built specifically for silicon verification. In modern chip projects, verification eats up more than 70% of the entire development cycle, and senior engineers spend most of their days manually scrolling through massive logs and waveforms. VeriSight automates that entire triage process — it pinpoints whether a failure is an RTL bug, a testbench bug, or a spec issue in under 15 seconds, and even drafts a verified code patch."

Q2. How is VeriSight different from existing EDA tools like Synopsys Verdi or Siemens Questa?

Delivery Cue: *Use the medical X-ray analogy; audiences immediately understand it.*

Spoken Answer: "Verdi and Questa are world-class debugging viewers — they give you waveform visualizers and signal tracing windows, but a human engineer still has to do 100% of the thinking and manual digging. VeriSight does not replace Verdi; it acts as the intelligent brain on top of it. It automatically reads the logs and waveform dumps, applies formal decision rules, isolates the exact root cause, and generates the fix. It’s the difference between looking at an X-ray yourself versus having an expert radiologist hand you the diagnosis instantly."

Q3. How did you arrive at the '100x faster triage' claim?

Delivery Cue: *State the 4-6 hours manual vs. 15 seconds AI comparison clearly.*

Spoken Answer: "We benchmarked real-world verification workflows. When a regression test fails on an SoC, an engineer typically downloads the log, opens the waveform, tracks the signal back across multiple clock domains, and checks the scoreboard — which takes on average 4 to 6 hours. VeriSight runs that entire multi-agent pipeline and outputs the root cause in 10 to 15 seconds. That represents a direct 100-fold reduction in manual triage latency."

Q4. Will VeriSight replace verification engineers?

Delivery Cue: *Reassure the audience that AI empowers engineers rather than replacing them.*

Spoken Answer: "Not at all. It elevates them. Chip design is growing so complex that there is a severe shortage of verification talent worldwide. Instead of having senior architects waste 60% of their workday doing repetitive log reading and chasing uninitialized reset bugs, VeriSight handles the routine triage so engineers can focus on high-value architecture, complex corner cases, and tapeout deadlines."

Q5. What is the business ROI for a semiconductor design house?

Delivery Cue: *Highlight the $15M-$30M mask re-spin cost.*

Spoken Answer: "The ROI comes from two huge factors: first, engineering time — saving thousands of man-hours per chip cycle. But second, and most importantly, preventing silicon mask re-spins. At advanced nodes like 3nm and 5nm, a single escaped bug that makes it to fab costs between $15M and $30M plus months of delay. Catching functional bugs before tapeout pays for the tool hundreds of times over."

2. Architecture & Multi-Agent Cognitive Fabric

Q6. Why did you use a 5-agent architecture instead of just one big prompt to an LLM?

Delivery Cue: *Emphasize modularity and how specialized agents prevent confusion.*

Spoken Answer: "Silicon verification requires completely different cognitive skills at each stage. One stage is log parsing, another is formal rule evaluation, another is hardware netlist tracing, another is report compilation, and another is code patching. If you try to do all of that in a single giant LLM prompt, the model gets confused, loses context, and hallucinates. By dividing the work into 5 specialized agents that communicate through structured JSON schemas, every agent performs its specific job with mathematical precision."

Q7. How do the agents pass data and communicate with each other?

Delivery Cue: *Mention Pydantic schemas and typed data contracts.*

Spoken Answer: "They communicate through strict Pydantic JSON contracts. Agent 1 creates a standardized `knowledge.json` containing the AST, tokenized logs, and sliced waveforms. Agent 2 reads that and produces `summary.json`. Agent 3 adds deep RTL analysis, Agent 5 writes the patch in `fix.json`, and Agent 4 compiles the final executive report. Because every handshake is strongly typed, no agent can pass corrupted or hallucinated data downstream."

Q8. What happens if an agent fails or an LLM call times out?

Delivery Cue: *Stress resilience and graceful fallback.*

Spoken Answer: "We built VeriSight with graceful fallback architecture. If an LLM API call fails or encounters network issues, the system automatically falls back to our deterministic offline rule engine. It continues executing Agent 3 and Agent 4 so the engineer still gets full lint analysis, VCD waveform slicing, and a comprehensive report with whatever evidence was collected, clearly flagging that LLM reasoning was unavailable."

Q9. Which LLMs are supported and how easy is it to switch between them?

Delivery Cue: *Mention that both cloud and local open-source models work interchangeably.*

Spoken Answer: "VeriSight has a pluggable model adapter. Out of the box, we support Google Gemini 3.8 Flash, Anthropic Claude 3.5 Sonnet, and OpenAI GPT-4o for cloud users. For air-gapped on-premise users, we connect directly via Ollama or vLLM to open-weights models like DeepSeek-Coder and Llama-3. You can switch models simply by changing an environment variable or selecting an option in our `./start` wizard."

Q10. Why did you choose Gemini 3.8 Flash and Claude Sonnet as cloud defaults?

Delivery Cue: *Highlight speed, large context, and SV code generation quality.*

Spoken Answer: "Gemini 3.8 Flash gives us extraordinary speed, huge context windows, and sub-second reasoning latency at pennies per run. Claude 3.5 Sonnet offers exceptional precision when synthesizing complex SystemVerilog syntax and temporal assertion logic. By supporting both, users can prioritize raw speed or deep formal synthesis depending on their regression needs."

3. Zero-Hallucination Guarantee & The 9 Decision Rules

Q11. Hallucination is the biggest fear in AI for hardware. How do you guarantee zero hallucinations?

Delivery Cue: *Speak firmly on the 'rule-grounded + sandbox-verified' principle.*

Spoken Answer: "We guarantee zero hallucinations through three strict guardrails: First, our 9 Formal Decision Rules restrict the AI from making ungrounded claims — every diagnosis must cite specific code lines, timestamps, and spec requirements. Second, our AST static analyzer checks code structure deterministically before the LLM is even consulted. And third, every proposed fix must compile and pass syntax checking in an isolated sandbox. If evidence is insufficient, the system is strictly mandated to classify as 'Unknown' rather than guess."

Q12. How were the 9 Decision Rules created?

Delivery Cue: *Sound authoritative — these are industry-standard failure taxonomies.*

Spoken Answer: "The 9 rules represent the core taxonomy of simulation failures formalized from industry standards. They systematically separate failures into Scoreboard Mismatch, X-Propagation, Protocol Handshake Violations, Timing/Clock Glitches, Testbench Driver Hangs, Memory Hazards, AST Lint issues, Simulation Timeouts, and Environment Mismatches. Each rule has explicit entry conditions and required evidence proofs."

Q13. How does the confidence score work?

Delivery Cue: *Explain that confidence is tied to physical evidence, not AI vibes.*

Spoken Answer: "Confidence is calculated using an evidence-weighted scoring model. If a diagnosis has multiple correlating proofs — for example, a scoreboard mismatch at timestamp T matching a spec latency clause and confirmed by signal state in the waveform — the confidence reaches 90-95%. If evidence is ambiguous or missing, the confidence score drops proportionally and alternative hypotheses are listed."

Q14. How does VeriSight tell the difference between an RTL bug and a Testbench bug?

Delivery Cue: *Use the RAM scoreboard latency example — it proves real-world depth.*

Spoken Answer: "This is one of VeriSight's strongest capabilities. It cross-checks the specification against both the RTL and the testbench. For example, in our RAM project test, the spec required a 1-clock-cycle synchronous read latency. The RTL correctly implemented the sequential register, but the UVM scoreboard was checking `rdata` combinationally in the very same cycle. VeriSight caught that the scoreboard was failing to model the pipeline latency, correctly isolating it as a Testbench Bug with 95% confidence."

Q15. What if the specification itself is wrong or ambiguous?

Delivery Cue: *Highlight that resolving spec ambiguities prevents team friction.*

Spoken Answer: "That is why Rule 9 and our 'Spec Bug' classification exist. If the specification contains contradictory requirements — for instance, one section specifies active-low reset while another requires active-high — VeriSight flags it as a 'Spec Bug' with exact clause references, preventing engineers from wasting days debating whether the bug belongs to RTL or Verification."

4. X-Tracer Engine & Waveform Analysis

Q16. What is the X-Tracer algorithm and how does it find uninitialized signals?

Delivery Cue: *Two layers: pyslang static AST finds reset-less registers; the VCD-backed x-tracer confirms the runtime path. Static ≠ simulation.*

Spoken Answer: "In digital design, an uninitialized flip-flop outputs an unknown state `X`, which quickly propagates through logic gates and corrupts the whole chip. X-Tracer works in two layers. (1) Static source analysis: we parse the RTL with pyslang (the Python bindings for the Slang IEEE-1800 SystemVerilog frontend) into a real elaborated AST, then flag sequential registers that are written but never assigned in their reset branch — these are the candidate X origins. (2) Simulation-based tracing (optional): when a VCD waveform is supplied, we drive the external x-tracer tool against a gate-level netlist; it steps back from the failing (signal, cycle) through the driving logic cone to the exact flop. The static layer identifies *candidate* origins from source alone; establishing the actual runtime propagation path requires the simulation layer, because propagation depends on dynamic scheduling and stimulus that source analysis cannot resolve."

Q17. Waveform dumps can be 10 to 50 Gigabytes. How do you handle them without running out of memory?

Delivery Cue: *Explain Temporal Waveform Slicing: only +/- 20 cycles around failure.*

Spoken Answer: "We never load an entire 50GB waveform into memory or pass it to an LLM. We use our high-speed Rust-based `pywellen` and `vcdvcd` parsers to perform 'Temporal Waveform Slicing'. We extract only the 20 clock cycles immediately surrounding the assertion failure timestamp, and only for the signals directly connected to the failing logic cone. That compresses gigabytes of trace data into a clean, lightweight JSON slice."

Q18. What if the simulation log doesn't include waveforms?

Delivery Cue: *Stress that the tool works with or without waveforms.*

Spoken Answer: "VeriSight is fully modular. If a VCD or FSDB waveform dump is provided, X-Tracer and waveform studio are activated. If only a simulation log and RTL/TB are provided, VeriSight continues seamlessly using log tokenization, AST inspection, and spec cross-referencing. It will simply inform the user that waveform evidence was not present."

Q19. Does X-Tracer support both synchronous and asynchronous reset styles?

Delivery Cue: *Demonstrate knowledge of synthesizable SV reset patterns.*

Spoken Answer: "Yes. X-Tracer analyzes the AST sensitivity list and conditional branches inside `always_ff` and `always` blocks. It checks whether `posedge` or `negedge` reset triggers are correctly matched with active-high or active-low conditions, flagging missing reset clauses or improper reset polarity immediately."

5. Security, Air-Gap & Enterprise IP Protection

Q20. Chip RTL is the crown jewel of our company. How do you prevent IP leakage?

Delivery Cue: *Speak with total conviction — security is the #1 dealmaker for chip firms.*

Spoken Answer: "Data privacy was our number one architectural priority. VeriSight provides two distinct enterprise security layers: First, **Sovereign Air-Gapped Mode**, where the entire platform runs on your on-premise compute cluster with zero internet access, using local models like DeepSeek-Coder. Second, if you do choose to use a cloud API, we have a built-in **RTL Anonymization Engine** that automatically strips proprietary module names, signal identifiers, and comments before sending prompts."

Q21. Can VeriSight run 100% offline in a classified or air-gapped lab?

Delivery Cue: *Clear, unambiguous: Yes, 100% offline.*

Spoken Answer: "Yes, 100%. We have tested and verified VeriSight running in completely disconnected environments with network adapters disabled. All AST parsing, VCD slicing, rule engines, local vector search, and local LLM inference run entirely on localhost."

Q22. What hardware is required to run VeriSight on-premise?

Delivery Cue: *Give concrete specs: standard PC for core, 1 GPU for local LLM.*

Spoken Answer: "For the deterministic rule engines, AST parsing, and reporting, any standard Linux x86 workstation or VM with 4 cores and 8GB RAM is plenty. If you wish to run a local 16-billion or 33-billion parameter model like DeepSeek-Coder locally, an enterprise workstation with an NVIDIA RTX 4090 or A100 GPU running Ollama or vLLM delivers instant, local inference."

Q23. Does VeriSight train on customer code or send telemetry back to you?

Delivery Cue: *Reassure customer confidentiality.*

Spoken Answer: "Never. VeriSight does zero model training and does not store customer RTL. In air-gapped mode, there is physically no outbound connection. When using commercial cloud APIs, we adhere to enterprise zero-data-retention API policies where customer inputs are never used for model training."

6. EDA Simulator Compatibility & CI/CD Regressions

Q24. How does VeriSight integrate with our existing simulation farm?

Delivery Cue: *Explain both Passive mode (just read logs) and Active mode (run sims).*

Spoken Answer: "VeriSight requires zero changes to your existing testbenches or Makefiles. It can operate in non-intrusive 'passive mode' by simply reading the simulation logs and VCD dumps your current regression produces. Or it can run in 'active mode' where our Closed-Loop Runner wraps simulator invocation directly."

Q25. Which commercial simulators are supported?

Delivery Cue: *List VCS, Questa, Xcelium, and Verilator.*

Spoken Answer: "All major Big-3 commercial engines: Synopsys VCS, Siemens QuestaSim/ModelSim, and Cadence Xcelium. It also fully supports open-source Verilator and Icarus Verilog. Our log parser has pre-trained tokenizers for VCS UVM error strings, Questa assertion reports, and standard `$display` outputs."

Q26. Can VeriSight run automatically in nightly CI/CD regression runs?

Delivery Cue: *Describe the automated CI workflow ending in Jira/Slack.*

Spoken Answer: "Yes. VeriSight features a complete headless CLI interface. You can add a single line to your Jenkins, GitLab CI, or GitHub Actions pipeline. If a test exits with non-zero status, VeriSight automatically wakes up, executes triage, generates `error.json` and `report.html`, and can post the diagnostic summary directly to a Slack channel or Jira bug ticket."

Q27. How does it handle multi-file UVM testbench hierarchies?

Delivery Cue: *Remind them that UVM packages and classes are fully linked.*

Spoken Answer: "Our UVM parser automatically inspects directory structures and resolves `` `include `` statements. When you point it to a testbench, it automatically gathers and parses all sibling classes — including the driver, monitor, scoreboard, sequence items, and virtual sequences — providing full hierarchical visibility."

7. Fix Synthesis, Sandbox Validation & Coverage

Q28. How does Agent 5 generate a code fix without introducing regression bugs?

Delivery Cue: *Emphasize minimal git diffs and sandbox verification.*

Spoken Answer: "Agent 5 operates under a strict minimal-code-region policy. It only modifies the lines directly responsible for the defect, keeping changes as small as possible. Most importantly, it creates a clean git patch and immediately compiles it in a temporary sandbox. If the patched code fails syntax checking or fails the regression, the fix is declined with full reasoning rather than presented blindly."

Q29. What happens if the engineer doesn't agree with the AI's proposed patch?

Delivery Cue: *Reinforce: 'Copilot, not autopilot; human in the loop'.*

Spoken Answer: "VeriSight is an engineer copilot, not an uncontrollable autopilot. The generated fix is saved as a standard unified `.patch` file alongside an interactive HTML diff viewer. The engineer can review the patch, adjust it, or discard it with a single click. The engineer always retains 100% final approval."

Q30. How does VeriSight help with Coverage Closure?

Delivery Cue: *Explain directed sequence generation for unhit coverpoints.*

Spoken Answer: "Closing the last 10 to 15% of functional coverage is notoriously difficult because standard random stimulus rarely hits deeply nested state machine corner cases. VeriSight's Plan Synthesizer analyzes unhit coverpoints in your UCDB database and automatically synthesizes directed UVM sequences specifically crafted to trigger those exact conditions."

Q31. Can VeriSight generate SystemVerilog Assertions (SVA) from scratch?

Delivery Cue: *Highlight formal SVA synthesis from English specs.*

Spoken Answer: "Yes. Agent 2 reads natural language specification clauses and translates them directly into formal SVA properties with correct implication operators (`|->` and `|=>`), clocking events, and reset disable qualifiers. This bridges the gap between text specs and executable formal checks."

8. Deployment, Onboarding & Future Roadmap

Q32. How long does it take for a new design team to deploy and onboard VeriSight?

Delivery Cue: *State 'less than 15 minutes, zero code instrumentation'.*

Spoken Answer: "It takes less than 15 minutes. VeriSight is delivered as a lightweight Python package or a ready-to-run Docker container. You clone the repo, run `./start`, configure your simulator path or API key, and you are immediately running triage on your existing test suites. There is zero proprietary compiler plugin or design instrumenting required."

Q33. What is the role of ChromaDB vector store in VeriSight?

Delivery Cue: *Describe ChromaDB as 'institutional memory for verification'.*

Spoken Answer: "ChromaDB acts as the team's long-term institutional memory. Every time VeriSight diagnoses and fixes a bug, it stores the error signature, root cause, and fix in a local vector database. When a similar failure happens 3 months later on a different block, VeriSight instantly recalls the previous solution, enabling instant cross-team knowledge sharing."

Q34. What are the key milestones on your upcoming product roadmap?

Delivery Cue: *List FSDB, Jira integration, and UCIe/CXL chiplet protocols.*

Spoken Answer: "Our immediate next milestones are: first, native support for Cadence FSDB waveform format via commercial fast-C APIs; second, automatic Jira and Bugzilla bi-directional ticket creation; and third, multi-chiplet protocol verification for UCIe and CXL interfaces."

Q35. How does VeriSight detect Clock Domain Crossing (CDC) and metastability bugs?

Delivery Cue: *Mention multi-flop synchronizers and async clock domains.*

Spoken Answer: "Inside Agent 3, our dedicated CDC engine traces signals crossing between asynchronous clock domains. It inspects whether multi-flop synchronizers, handshake synchronizers, or asynchronous FIFOs are implemented. If data is read across domains without adequate synchronization, it immediately flags the crossing with source and destination clock domain tags and suggests inserting standard double-flop synchronizers."

Q36. Can VeriSight scale to massive, multi-million gate SoC designs?

Delivery Cue: *Emphasize hierarchical partitioning rather than flattening.*

Spoken Answer: "Yes. VeriSight scales through hierarchical AST partitioning. Rather than flattening an entire 500-million gate SoC, Agent 1 maps the top-level block hierarchy and focuses the deep cone analysis specifically on the failing module and its immediate interface boundaries. This keeps memory footprint low and execution times sub-minute regardless of full chip size."

Q37. What is the false-positive rate of your diagnostic engine?

Delivery Cue: *Stress the <3% rate and the multi-stage consensus filter.*

Spoken Answer: "In our regression benchmark suites, the false-positive rate is under 3%. This is because of our multi-stage consensus filter: a root cause is only accepted if the log error signature, the AST code structure, and the 9-rule formal engine all agree. If there is any doubt or contradiction, the engine deliberately outputs 'Unknown' with alternative hypotheses rather than giving a false lead."

Q38. How does VeriSight support Functional Safety standards like ISO 26262?

Delivery Cue: *Mention automotive safety and ASIL-D traceability.*

Spoken Answer: "For automotive and mission-critical silicon, Agent 2 generates complete bidirectional traceability matrices. It links every architectural safety requirement directly to its corresponding SystemVerilog assertion and coverage bin. This gives verification teams an audit-ready compliance trail showing 100% verification closure for ASIL-D safety goals."

Q39. How does VeriSight integrate with Git and review workflows like GitHub or Gerrit?

Delivery Cue: *Describe the seamless Pull Request / Gerrit workflow.*

Spoken Answer: "Agent 5 outputs clean, standardized unified git diff patches. VeriSight can be configured to automatically open a pull request on GitHub or push a change to Gerrit with the full diagnostic report and test logs attached. Engineering leads can inspect the before-and-after diff, see the passing sandbox run, and approve the merge with one click."

Q40. If you had to summarize VeriSight in one closing sentence, what would it be?

Delivery Cue: *Deliver with passion, conviction, and a confident closing smile.*

Spoken Answer: "VeriSight transforms silicon verification from a slow, painful multi-week debugging bottleneck into an automated, 15-second closed-loop copilot — empowering chip teams to tape out faster, with lower costs, and zero hallucinations."